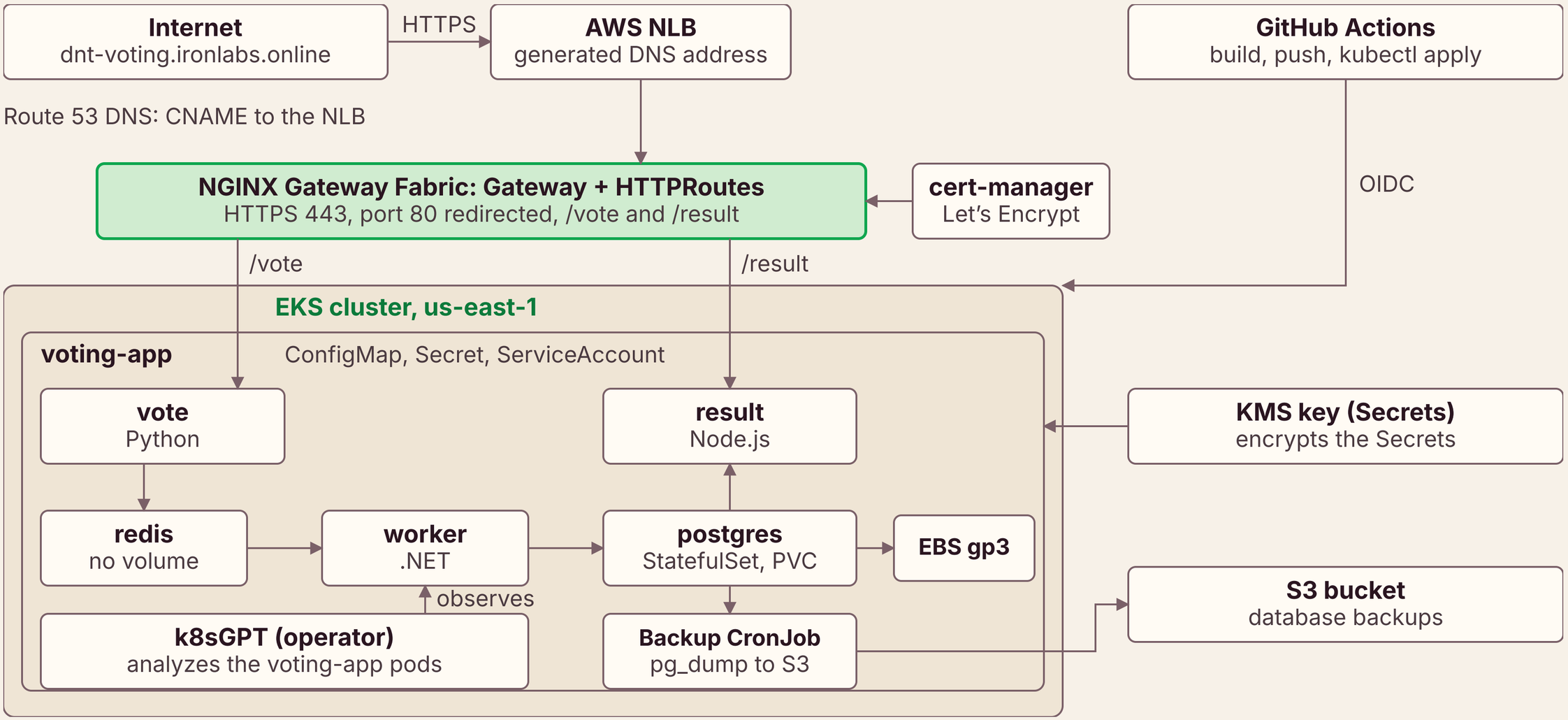
IRONHACK · PROJECT 2

Building, Securing and Exposing a Voting Application on Amazon EKS

The Voting App deployed on Amazon EKS, with a GitHub Actions pipeline

Dally NTOLO · October 2026

One Public Entry Point, Private Backend Services



THE GOAL

One Platform

Secure EKS Infrastructure with Automated CI/CD

Vote	Python, the ballot
Redis	the vote queue
Worker	.NET, the carrier
Postgres	the results database
Result	Node.js, the final tally

Project Requirements: A Detailed Breakdown

EKS cluster	eksctl, 2 t3.medium nodes, OIDC enabled	Done
Redis and Postgres	Redis Deployment, Postgres StatefulSet with a gp3 volume	Done
vote, result, worker	Deployments and Services, internal DNS	Done
/vote /result routing	Gateway API (NGINX Gateway Fabric) instead of Ingress	Done
Secrets	Secret and ConfigMap injected with envFrom	Done
CI/CD	GitHub Actions: build, push to Docker Hub, kubectl apply	Done
HTTPS	cert-manager, Let’s Encrypt, Route 53, domain ironlabs.online	Done

PIPELINE

Git Push → Automatic Deployment

1
Git Push
to the main branch

2
Build
three images in parallel

3
Publish
Docker Hub, tag = commit SHA

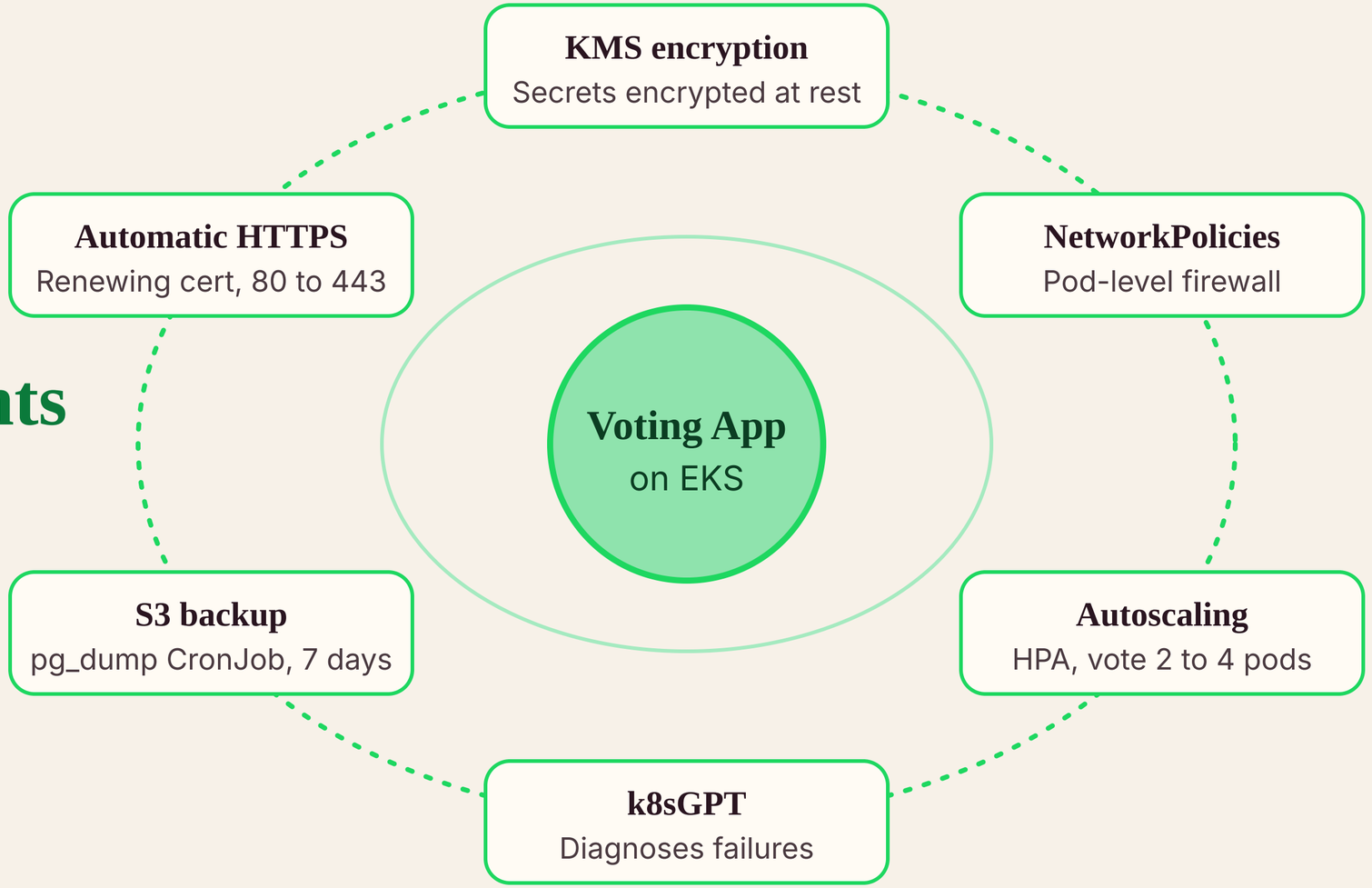
4
Sign in
AWS via OIDC, no stored key

5
Deploy
kubectl apply, rollout status

Images built: vote · result · worker

Least-Privilege IAM Access to the voting-app Namespace

Extra Improvements



Verified and Working

Green pipeline

A push updates the app, image tagged with the SHA.

Valid HTTPS

Let’s Encrypt certificate issued for dnt-voting.ironlabs.online.

Vote end to end

vote, Redis, worker, Postgres then result: the count moves.

Working firewall

Allowed flows pass. Forbidden flows are blocked.

Failure diagnosed

Missing image nginx:9.99.99: k8sGPT spots the ErrImagePull error.

Backup in S3

The Postgres dump lands in the private bucket.

WHAT DID NOT WORK

Locked Out by a Mismatched Identity

SYMPTOM

Access denied

The pipeline fails: “Not authorized to perform sts:AssumeRoleWithWebIdentity”.

1 Investigation

A debug workflow prints the real “sub” of the GitHub token, which did not look like the value expected in the trust policy.

2 Fix

Policy rewritten with the right sub (StringEquals). The ARN from the command line did not work: I copied the role ARN straight from the AWS console into the AWS_ROLE_ARN secret.

Lessons Learned and Next Steps

Major lessons

- Read the evidence (the OIDC token) rather than guess
- Isolate each layer: namespace, ServiceAccount, NetworkPolicy
- Everything in Git, no secret inside
- Check before acting: git log, kubectl describe, logs

Next steps

- Plug a real AI backend into k8sGPT
- Block double voting in the application
- Clean up in order: DNS, Gateway, volumes, cluster, IAM, S3, KMS

Thank you for your attention

Questions are welcome.

Vote: https://dnt-voting.ironlabs.online/

Results: https://dnt-voting.ironlabs.online/result/